

Ogunsola Toluwase Paul

Backend Software Engineer | Node.js | TypeScript | Python

Oyo State, Nigeria | Remote (Nigeria-based)

oluwayeyiogunsola90@gmail.com | +234 806 700 4668 | [GitHub](#) | [LinkedIn](#) | [Portfolio](#)

SUMMARY

Backend-focused software engineer specializing in Node.js/TypeScript and Python, with hands-on experience building and shipping applications involving financial integrity, concurrency, authorization, asynchronous processing, and failure recovery. Experienced with PostgreSQL, Redis, BullMQ, REST APIs, authentication, and distributed coordination, with a strong focus on reliability and correctness. Currently pursuing a B.Sc. in Computer Science; available to work full-time.

EXPERIENCE

Freelance Web Developer — Local Business Client

Jun 2025 – Aug 2025

Remote | Paid

- Built and launched a booking platform for a local salon, replacing phone-based scheduling with an online customer booking workflow.
- Designed and implemented a Node.js/Express REST API backed by PostgreSQL for appointment and customer data.
- Owned the complete delivery lifecycle independently, from requirements gathering and implementation through deployment, debugging, and maintenance.
- Deployed the React frontend to Vercel and backend to Railway, configuring a custom domain and SSL for production.
- Maintained the production application for 3 months post-launch, fixing bugs and implementing requested features.

Technologies: Node.js, Express, PostgreSQL, React, Vercel, Railway

PROJECTS

Two production-grade systems designed and built independently, deployed and live.

Kith — Family Financial Coordination Platform

Node.js | TypeScript | Express | PostgreSQL | Redis | BullMQ | React | Supabase | Firebase | Resend

GitHub: github.com/Devpaul-01/Kith | Live Demo: kith-self.vercel.app

Multi-tenant financial coordination platform with 199 API endpoints and ~25–30 core tables, designed around financial correctness, concurrency, background processing, authorization, and auditability.

- **Financial integrity:** Append-only ledger where confirmed contributions can't be edited in place; corrections create linked entries preserving original history, plus idempotency keys and a 10-minute duplicate-submission detector.
- **Concurrency and atomicity:** Moved race-sensitive multi-step workflows into PostgreSQL atomic RPCs (dispute resolution, invite acceptance, contributor-target updates, event-to-recurring conversion), eliminating check-then-write races.
- **Auditable disputes:** Admin-adjudicated dispute workflow with timestamped notes, mandatory resolution explanations, atomic status transitions, notifications, and linked financial corrections.
- **Asynchronous processing:** 9-queue BullMQ architecture with per-queue concurrency controls, recurring schedules, retries, and a PostgreSQL-backed notification outbox with recovery scanning.
- **Multi-tenant authorization:** JWT → workspace membership → role/ownership authorization with enumeration-resistant 404s, plus a 30-second Redis membership cache with active invalidation.
- **Testing and CI:** Unit, integration, and rate-limit tests covering financial state transitions, duplicate detection, authorization, and abuse-prevention behavior; CI runs against real PostgreSQL and Redis.
- **Architecture:** 18 Architecture Decision Records covering concurrency, consistency, authorization, caching, and reliability trade-offs.

StudyHub — Real-Time Learning Platform

Python | Flask | SQLAlchemy | PostgreSQL | Redis | WebSockets | Multi-Provider AI

GitHub: github.com/Devpaul-01/Studyhub | Live Demo: studyhub-two-psi.vercel.app

- **Multi-instance correctness:** Replaced in-memory WebSocket presence, rate limits, and scheduler state with Redis for safe multi-worker deployment. Scheduler uses a fail-closed distributed lock (atomic SET NX EX + Lua compare-and-delete release) — the app's only fail-closed Redis consumer, since duplicate snapshots corrupt data but a skipped tick is cheap.
- **Authentication security:** JWT access tokens with DB-backed rotating refresh tokens hashed at rest, family-wide revocation on reuse detection, and a grace window distinguishing stolen-token replay from an ordinary multi-tab refresh race. Google OAuth anchored on the provider's account ID rather than email, closing an account-hijack path.
- **AI provider resilience:** Six-provider LLM failover layer with a four-way failure taxonomy (KEY_FAULT, PROVIDER_TRANSIENT, BAD_MODEL, NON_RETRYABLE), so an outage no longer cools a healthy API key. Health state shared across instances via Redis behind a fail-open kill switch, with mid-stream provider switching.
- **AI latency isolation:** Thread @mention AI dispatch runs on a bounded 8-worker ThreadPoolExecutor rather than the RQ queue used for email — a lost AI reply is a bounded UX gap, unlike a lost transactional email. Keeps slow provider calls from blocking the WebSocket event loop without unneeded queue durability.
- **Architecture & testing:** CI-enforced services/routes boundary via AST static analysis, so background workers and WebSocket handlers call the same Flask-free service functions as HTTP routes. Unit + integration suites (pytest) on GitHub Actions, with distributed-lock atomicity tested against real Lua execution (fakeredis + lupa).

TECHNICAL SKILLS

- **Languages:** TypeScript, JavaScript, Python
- **Backend:** Node.js, Express.js, Flask, REST APIs
- **Data and Systems:** PostgreSQL, SQL, MongoDB, SQLAlchemy, Redis, BullMQ, asynchronous processing, distributed coordination
- **Testing and Security:** Jest, Pytest, CI, JWT, OAuth, refresh-token rotation, CSRF, rate limiting, RBAC
- **Infrastructure:** Docker, Git, GitHub, Supabase, Vercel, Railway, AWS
- **AI and Real-Time:** LLM orchestration, provider failover, WebSockets, Socket.IO

EDUCATION

Federal University of Technology, Akure (FUTA) — B.Sc. Computer Science

Currently enrolled — coursework permits full-time work